model.py

Credit Card Fraud Detection — Random Forest + SMOTE

■ What This File Does

This script trains a machine learning model to classify credit card transactions as **legitimate (0)** or **fraudulent (1)**. Because real-world fraud datasets are extremely imbalanced (typically <0.2% fraud), the script uses **SMOTE** to synthetically balance the training set before fitting a **Random Forest** classifier. Two diagnostic plots are saved as PNG files.

■ Full Source Code

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (classification_report, confusion_matrix,
                             precision_recall_curve, auc)
import matplotlib.pyplot as plt
import seaborn as sns
import os
try:
    from imblearn.over_sampling import SMOTE
except ImportError:
    print("imbalanced-learn not installed, will skip SMOTE if it fails")

DATA_FILE = "creditcard.csv"

def load_data():
    if not os.path.exists(DATA_FILE):
        raise FileNotFoundError(f"{DATA_FILE} not found.")
    df = pd.read_csv(DATA_FILE)
    return df

def train_and_evaluate():
    df = load_data()
    df = df.dropna()

    X = df.drop('Class', axis=1)
    y = df['Class']

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y)

    try:
        smote = SMOTE(random_state=42)
```

```
        X_train_res, y_train_res = smote.fit_resample(X_train, y_train)
    except NameError:
        X_train_res, y_train_res = X_train, y_train

    model = RandomForestClassifier(n_estimators=50, random_state=42, n_jobs=-1)
    model.fit(X_train_res, y_train_res)

    predictions = model.predict(X_test)
    probs = model.predict_proba(X_test)[:, 1]

    print(classification_report(y_test, predictions))

    cm = confusion_matrix(y_test, predictions)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.savefig('confusion_matrix.png')

    precision, recall, _ = precision_recall_curve(y_test, probs)
    pr_auc = auc(recall, precision)
    plt.plot(recall, precision, label=f'PR AUC = {pr_auc:.2f}')
    plt.savefig('precision_recall_curve.png')

if __name__ == "__main__":
    train_and_evaluate()
```

■ Block-by-Block Code Explanation

1. Imports

pandas, numpy

pandas loads and manipulates the CSV as a DataFrame. numpy provides array math used internally by scikit-learn and SMOTE during resampling.

train_test_split

Randomly splits the dataset into training and test sets while preserving the fraud/legit ratio (stratify=y). This prevents a split where all fraud cases accidentally end up in training only.

RandomForestClassifier

An ensemble model that builds many decision trees in parallel, each trained on a random subset of data and features. Final prediction = majority vote across all trees. Very robust against overfitting.

classification_report, confusion_matrix, precision_recall_curve, auc

Evaluation tools. classification_report prints precision, recall and F1 per class. confusion_matrix shows TP/FP/TN/FN counts. precision_recall_curve and auc measure the quality of probability predictions — critical for imbalanced datasets.

SMOTE (try/except)

Imported inside a try/except so the script still runs if imbalanced-learn is not installed — it just skips resampling. SMOTE = Synthetic Minority Oversampling Technique.

2. load_data()

os.path.exists(DATA_FILE)

Fails loudly with a FileNotFoundError if creditcard.csv is missing, rather than letting pandas raise a confusing low-level error. Gives the user a clear message to download the data first.

pd.read_csv(DATA_FILE)

Loads the entire ~285,000-row dataset into memory as a DataFrame. Columns are: Time, V1–V28 (PCA-transformed features), Amount, Class.

3. Data Preparation inside train_and_evaluate()

df.dropna()

Removes any rows containing NaN/null values to prevent errors during model training. The creditcard dataset is clean, so this is a safety measure.

X = df.drop('Class', axis=1)

Creates the feature matrix X by removing the target column. X has shape ~(285000, 30) — 30 input features per transaction.

y = df['Class']

The target vector. 0 = legitimate transaction, 1 = fraud. In this dataset ~99.83% are 0 and only ~0.17% are 1 — a severe imbalance.

train_test_split(..., test_size=0.2, stratify=y)

Splits data: 80% → training (~228,000 rows), 20% → test (~57,000 rows). stratify=y ensures both splits contain the same ~0.17% fraud ratio, so the test set has real frauds to evaluate against.

4. SMOTE — Handling Class Imbalance

The training set after splitting has roughly 227,451 legitimate vs 394 fraud transactions. A model trained on this raw data will simply predict 'legitimate' for everything and achieve 99.8% accuracy while missing all fraud — useless.

smote = SMOTE(random_state=42)

Creates a SMOTE resampler. SMOTE works by: (1) picking a fraud sample, (2) finding its K nearest fraud neighbours in feature space, (3) generating synthetic fraud samples along the line between them. It does NOT just copy existing samples — it creates new, plausible ones.

X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

After resampling, X_train_res has ~454,902 rows: 227,451 legitimate + 227,451 synthetic frauds. The model now trains on a perfectly balanced dataset.

5. Training the Random Forest

RandomForestClassifier(n_estimators=50, random_state=42, n_jobs=-1)

n_estimators=50 → builds 50 decision trees. More trees = better accuracy but slower training. random_state=42 → sets the seed for reproducibility. n_jobs=-1 → uses ALL available CPU cores to train trees in parallel — much faster on multi-core machines.

model.fit(X_train_res, y_train_res)

Trains all 50 trees on the SMOTE-resampled balanced dataset. Each tree sees a random bootstrap sample of rows and a random subset of features at each split. This randomness is what makes Random Forests robust and prevents overfitting.

6. Prediction & Evaluation

predictions = model.predict(X_test)

Each tree votes 0 or 1 for each test row. The majority vote wins. Returns hard class labels (0 or 1) for all ~57,000 test rows.

probs = model.predict_proba(X_test)[:, 1]

Returns the probability of fraud (class 1) for each row. [:, 1] selects the second column (fraud probability). Used to draw the Precision-Recall curve at different decision thresholds.

classification_report(y_test, predictions)

Prints per-class precision, recall, F1-score and support. For fraud detection, recall for class 1 is the most important — it tells us what fraction of actual fraud cases we caught.

7. Confusion Matrix Plot

A 2×2 grid showing: True Negatives (top-left), False Positives (top-right), False Negatives (bottom-left), True Positives (bottom-right). Saved as confusion_matrix.png.

True Positive (TP)	Fraud correctly predicted as fraud — what we want to maximise
True Negative (TN)	Legit correctly predicted as legit
False Positive (FP)	Legit flagged as fraud — customer inconvenience
False Negative (FN)	Fraud missed — most dangerous outcome

8. Precision-Recall Curve

Plots Precision (of flagged transactions, what % are actually fraud) vs Recall (of all fraud, what % did we catch) at every possible threshold. A high PR-AUC score (close to 1.0) means the model ranks fraud highly. Preferred over ROC-AUC for imbalanced datasets. Saved as precision_recall_curve.png.